

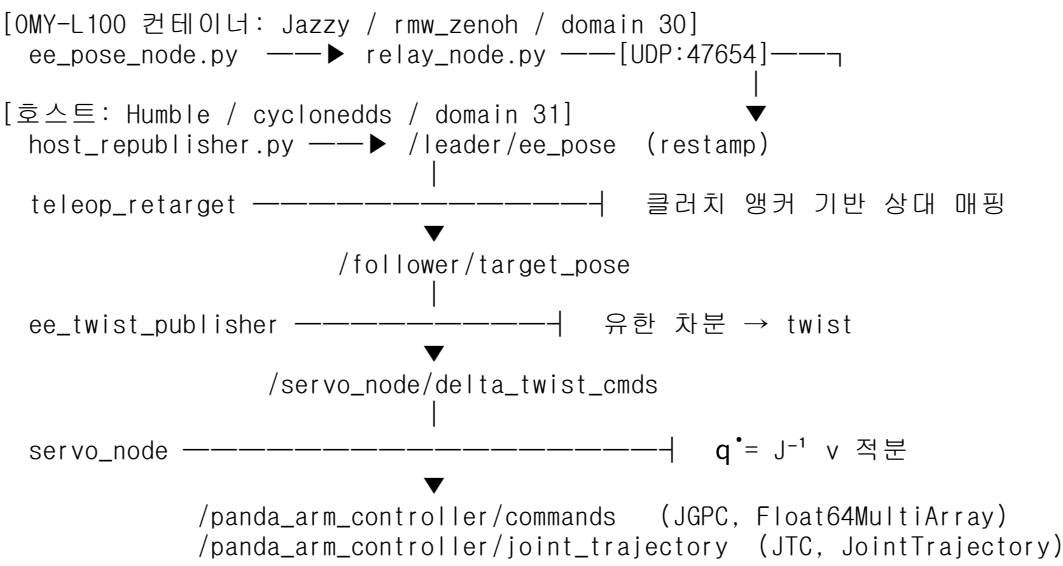
OMY-L100 리더로 팔로워 로봇 제어하기 — 실행 절차

teleop 문서 진입점: [README.md](#) — 하려는 일부터 문서를 고른다.

OMY-L100 리더 암을 사람이 움직이면 rdfp 팔로워 로봇(rdfp_panda_jgpc_mock / rdfp_panda_mock)이 그 동작을 따라 하게 만드는 **운영 절차서** 다.

- 계약(토픽·QoS·stamp 규약): [external_input_adapters.md](#)
- 설계 배경(retarget 수식, 클러치 원리): [leader_follower_mirroring_design.md](#)
- 리더 어댑터 구현: 외부 저장소 omy_leader_bridge

1. 전체 체인



리더의 /tf 는 rdfp 그래프에 들어오지 않는다. **브릿지가** /leader/ee_pose 를 직접 공급하므로 rdfp 는 리더 TF 를 알 필요가 없다.

실측 유량 (검증 완료)

토픽	주기	비고
/leader/ee_pose	200 Hz 발행 기준. 실효 19 Hz (§ 5-4)	
/follower/target_pose	19 Hz	최대 변위 0.259 m
/servo_node/delta_twist_cmds	19 Hz	
/panda_arm_controller/commands	29 Hz	servo publish_period 0.034s
/joint_states	—	최대 편차 0.600 rad (34.4°)

2. 사전 확인

2-1. ROS 환경

모든 터미널에서 먼저 실행한다. 이게 어긋나면 노드가 서로 안 보이고, 증상은 "토픽이 아예 없다" 라 원인을 짚기 어렵다.

```
source ~/.ros2rc # ROS_DOMAIN_ID=31 + cyclonedds
source ~/development/ros/rdfp_ws/install/setup.bash
```

~/.ros2rc 는 ~/.bashrc 가 자동으로 부르지 않는다. 확인:

```
echo "domain=${ROS_DOMAIN_ID:-0} rmw=${RMW_IMPLEMENTATION:-fastrtps}"
# domain=31 rmw=rmw_cyclonedds_cpp 이어야 한다
```

2-2. 기존 스택 정리

이전 스택이 /controller_manager 를 잡고 있으면 컨트롤러가 아예 안 뜬다.

```
ps -eo pid,pgid,args | grep "ros2 launch rdfp" | grep -v grep
kill -TERM -<PGID> # 프로세스 그룹째
```

pkill 은 launch 부모만 죽이고 자식을 systemd --user 로 남긴다. 남은 고아가 있으면 다음 기동이 조용히 실패하므로 반드시 그룹 단위로 종료한다.

```
ps -eo pid,ppid,args | grep install/rdfp/lib/rdfp | grep -v grep # 고아 확인
ros2 daemon stop # 캐시 갱신
```

3. 실행 절차

터미널 1 — 팔로워 스택

```
ros2 launch rdfp rdfp_panda_jgpc_mock.launch.py
```

컨트롤러 타입을 확인한다.

```
ros2 control list_controllers
# joint_state_broadcaster joint_state_broadcaster/JointStateBroadcaster active
# panda_arm_controller position_controllers/JointGroupPositionController active
# panda_hand_controller position_controllers/GripperActionController active
```

panda_arm_controller 가 joint_trajectory_controller 로 보이면 이전 스택 잔재다 (§ 2-2 로 돌아간다).

JGPC 스택은 servo 출력을 /panda_arm_controller/commands (std_msgs/Float64MultiArray) 로 override 하므로, move_group plan & execute 는 안 되지만 servo 경로는 정상 동작한다. 본 절차는 servo 경로만 쓴다.

터미널 2 — 리더 브릿지

```
cd ~/development/ros/omy_leader_bridge
./bridge.sh start
./bridge.sh status
```

bridge.sh start 는 host republisher → ee_pose(컨테이너) → relay(컨테이너) 순으로
떠운다. UDP 는 수신자가 없으면 패킷이 조용히 버려지므로 **호스트가 먼저 bind 해야
한다.**

리더 스트림 확인:

```
ros2 topic hz /leader/ee_pose          # 약 50 Hz
ros2 topic echo /leader/ee_pose --once
```

터미널 3 — 미러링 체인

```
ros2 launch rdfp teleop_mirror.launch.py
```

본 런치는 /leader/ee_pose 의 발행자를 만들지 않는다. 터미널 2의 브릿지가
공급한다는 전제다. 따라서 브릿지를 먼저 띄워야 하며, 브릿지가 죽으면 클러치가
자동 해제된다 (§ 5-1).

작업공간 제한과 정렬을 함께 주는 것을 권한다.

```
ros2 launch rdfp teleop_mirror.launch.py W
  workspace_min:="[0.1, -0.5, 0.05]" workspace_max:="[0.8, 0.5, 0.9]" W
  position_scale:=1.0 align_yaw:=0.0
```

인자

용도

workspace_min / workspace_max	목표 pose 를 박스 안으로 clamp — 도달 불가 영역 진입 방지
position_scale	리더/팔로워 작업공간 크기 비율 보정
align_yaw	리더와 팔로워의 "앞" 방향 정렬. pi 면 좌우 반전
lpf_cutoff_hz	손떨림 제거 (기본 3.0)
pedal_timeout	USB 풋페달 데드맨 (기본 0 = 비활성). 아래 참고

USB 풋페달로 조작하려면

클러치를 서비스 호출 대신 발로 밟아 잡을 수 있다. **밟는 동안만 engage** 되는
데드맨 구성이며, 터미널 4의 클러치 명령이 필요 없어진다.

터미널 3을 이렇게 바꾸면 페달 노드까지 함께 뜬다.

```
ros2 launch rdfp teleop_mirror.launch.py W
  enable_clutch_pedal:=true W
  pedal_device_name:=pedal pedal_key_code:=KEY_A
```

pedal_timeout(하트비트 감시)은 런치가 자동으로 채운다 — 페달 노드가 죽거나 USB 가 빠졌을 때 클러치가 물린 채 남는 것을 막는 장치다.

python3-evdev 설치, input 그룹 권한, 장치명·키코드 확인 방법은 [clutch_pedal_guide.md](#) 참고. 페달 노드를 따로 띄우려면 같은 문서의 "방법 B" 를 본다.

기동 노드는 /teleop_retargert 과 /ee_twist_publisher 둘뿐이어야 한다.

```
ros2 node list | grep -E "teleop_retargert|ee_twist"
```

터미널 4 — servo 시작 → 클러치 engage

순서를 지킨다.

```
# 1) servo 기동
ros2 service call /servo_node/start_servo std_srvs/srv/Trigger

# 2) 리더 스트림 확인 (3초 측정 후 자동 종료)
timeout 3 ros2 topic hz /leader/ee_pose
# → average rate: 49.997

# 3) 클러치 engage
ros2 service call /teleop_retargert/clutch std_srvs/srv/SetBool "{data: true}"
# → success=True, message='engaged'
```

rdfp_panda_jgpc_mock.launch.py 에는 servo_auto_start_node 가 포함되어 있지 않으므로 start_servo 는 수동 호출해야 한다. 빠뜨리면 servo 가 모든 twist 를 경고 없이 버린다.

리더 스트림 확인 방법 두 가지

engage 호출 자체가 검사다. teleop_retargert 은 요청을 받으면 양쪽 pose 의 신선도를 확인하고 실패 시 사유를 돌려준다. 따로 확인하지 않고 바로 호출해도 어느 쪽이 문제인지 알 수 있다.

응답	의미
success=True, message='engaged'	리더·팔로워 pose 둘 다 신선 → 정상
success=False ... leader pose is missing or stale	리더 스트림 문제 (터미널 2 확인)
success=False ... follower pose is missing or stale	팔로워 스택의 /ee_pose 문제 (터미널 1 확인)

그럼에도 topic hz 를 먼저 보는 이유 — 두 검사의 기준이 다르다.

시점	파라미터	기본값	검사 내용
engage 시	pose_staleness	1.0초	최근 1초 안에 메시지가 왔는가
engage 후 감시	watchdog_timeout	0.5초	0.5초 넘게 끊기면 자동 해제

engage 는 "최근 메시지 존재" 만 보므로 **1 Hz 로 띄엄띄엄 오는 스트림도 engage 는 성공한다.** 그러나 곧바로 watchdog 에 걸려 해제된다.

```
[teleop_retarget] Clutch disengaged: leader pose stream stale for 0.55s
```

topic hz 는 **주기까지** 알려주므로 이 상황을 미리 잡는다. 계약상 실패 10 Hz 이상이어야 하고 30 Hz 를 권장한다 (§ 5-4).

해제:

```
ros2 service call /teleop_retarget/clutch std_srvs/srv/SetBool "{data: false}"
```

리더를 편한 자세로 되돌린 뒤 다시 engage 하면 그 시점이 새 앵커가 된다 (마우스를 들어 옮기는 것과 같다).

현재 클러치 상태 확인

```
ros2 topic echo /teleop_retarget/clutch_state --once          # 현재 상태
ros2 topic echo /teleop_retarget/clutch_state                # 변화 감시
ros2 service call /teleop_retarget/get_clutch_state std_srvs/srv/Trigger # 동기 조회
```

셋 다 읽기 전용이라 상태를 바꾸지 않는다. 토픽은 TRANSIENT_LOCAL 이라 늦게 구독해도 현재 상태를 즉시 받고, **자동 해제를 폴링 없이 감지한다.**

```
engaged: false
reason: leader pose stream stale for 0.60s
```

~/clutch 를 조회 용도로 부르면 안 된다 — SetBool 이라 disengaged 일 때 {data: true} 를 부르면 그대로 engage 되어 버린다.

4. 동작 확인

가장 확실한 것은 관절값을 직접 보는 것이다. RViz 로는 느린 움직임을 놓치기 쉽다.

```
ros2 topic echo /joint_states --field position
```

체인이 어디서 끊겼는지 찾을 때:

```
for t in /leader/ee_pose /follower/target_pose \
  /servo_node/delta_twist_cmds /panda_arm_controller/commands; do
  echo "--- $t"; timeout 4 ros2 topic hz $t
done
```

끊긴 지점

/leader/ee_pose 없음

/follower/target_pose 는 나오는데 **값이 안 변함** 클러치 disengaged (§ 5-1)

/servo_node/delta_twist_cmds 없음

/panda_arm_controller/commands 없음

원인

브릿지 미기동, 또는 ROS 환경 불일치 (§ 2-1)

teleop_mirror 미기동

start_servo 미호출 (§ 5-2)

5. 문제 해결

5-1. 토픽은 흐르는데 팔이 안 움직인다 — 클러치 자동 해제

가장 흔하다. 리더 스트림이 `watchdog_timeout`(기본 0.5초) 넘게 끊기면 `teleop_retarget` 이 **자동으로 disengage** 하고, 이후 마지막 목표 `pose` 를 계속 재발행(hold)한다.

유량은 정상인데 값만 안 변하므로 동작 중인 것처럼 보인다.

```
[teleop_retarget] Clutch disengaged: leader pose stream stale for 0.55s
```

로그를 못 보는 상황이면 상태 토픽·서비스가 같은 사유를 돌려준다.

```
ros2 topic echo /teleop_retarget/clutch_state --once
# → engaged: false / reason: leader pose stream stale for 0.50s
```

터미널 3 로그에서 이 줄을 확인하고 다시 `engage` 한다. 리더 통신이 자주 끊기면 `watchdog_timeout` 을 늘린다.

engage 는 성공했는데 곧바로 해제된다면 리더 주기가 너무 낮은 것이다. `engage` 판정 기준(`pose_staleness` 1.0초)이 `watchdog`(0.5초)보다 느슨해서, 2 Hz 미만 스트림은 `engage` 만 통과하고 유지되지 못한다 (§ 3 터미널 4 참고).

```
timeout 3 ros2 topic hz /leader/ee_pose      # 주기부터 확인
```

`max_sample_jump`(기본 0.2 m)를 넘는 급격한 이동도 자동 해제 사유다 — 통신 복구 직후 값이 튀는 것을 막기 위한 장치다.

5-2. start_servo 누락

`moveit_servo` 는 `start_servo` 호출 전까지 **모든 입력을 조용히 버린다**. 에러도 경고도 없다.

```
ros2 service call /servo_node/start_servo std_srvs/srv/Trigger
ros2 topic echo /servo_node/status --once      # data: 0 (NO_WARNING) 이면 정상
```

5-3. ROS 환경 불일치

`ros2 daemon` 프로세스가 두 개 뜨면 환경이 갈린 것이다.

```
$ ps -eo args | grep ros2-daemon | grep -v grep
... --ros-domain-id 0 --rmw-implementation rmw_fastrtps_cpp      ← 잘못된 쪽
... --ros-domain-id 31 --rmw-implementation rmw_cyclonedds_cpp  ← 정상
```

스택 프로세스의 실제 환경도 직접 볼 수 있다.

```
tr 'W0' 'Wn' < /proc/<launch_pid>/environ | grep -E "ROS_DOMAIN_ID|RMW_IMPL"
```

해결은 터미널의 변수를 지우는 것이 아니라 스택을 31 로 다시 띄우는 것이다.

5-4. 움직임이 예상보다 작다 / 거칠다

리더의 실효 발행 주기를 확인한다. ee_twist_publisher 는 dt <= 0 인 중복 stamp 표본을 버리므로, 발행 주기가 높아도 실효 주기는 낮을 수 있다.

```
ros2 topic hz /leader/ee_pose          # 발행 주기
ros2 topic hz /servo_node/delta_twist_cmds # 실효 주기
```

실효 주기

상태

< 10 Hz servo stale → 끊김. 개루프라 감속분이 영구 오차

10 ~ 29 Hz 동작하나 servo 주기(29.4 Hz)를 못 채워 계단식

≈ 30 Hz 이상 정합

움직임 크기 자체는 게인으로 조정한다.

```
ros2 launch rdfp teleop_mirror.launch.py W
twist_linear_gain:=5.0 twist_angular_gain:=2.5
```

기본값 2.5 / 1.25 는 servo 의 command_in_type: unitless

(scale.linear=0.4, scale.rotational=0.8)를 정확히 상쇄해 원본 속도 1.0배 를 만드는 값이다. 올리면 그만큼 리더 동작이 과장된다.

6. 리더 없이 시험하기

OMY-L100 이 없을 때 녹화된 bag 을 리더 대역으로 쓸 수 있다. 터미널 2(브릿지)를 아래로 대체한다.

```
ros2 bag play data/ee_pose_bag -l --remap /leader/ee_pose_states:=/leader/ee_pose
```

-l (반복)을 반드시 붙인다. 없으면 51.6초 뒤 스트림이 끊겨 클러치가 자동 해제된다 (§ 5-1).

-r 로 배속을 올리지 않는다. 속도는 stamp 기준으로 계산되어 배속과 무관한데 재생 지속시간만 짧아지므로, 총 이동량이 오히려 반비례로 줄어든다. 상세는 [../replay/replay_mock_stack_guide.md](https://replay.replay_mock_stack_guide.md) § 3-4-1.

bag 은 3일 전 stamp 를 그대로 내보내므로 실제 브릿지와 달리 restamp 되지 않는다. ee_twist_publisher 가 출력 stamp 를 발행 시각으로 갈아끼우므로 servo 까지는 문제없지만, 계약 (§ 3-2)을 만족하지 않는 시험용 구성임을 인지한다.

7. JTC 스택(rdfp_panda_mock)과의 차이

절차는 동일하고 **servo 출력 토픽만 다르다.**

	rdfp_panda_jgpc_mock	rdfp_panda_mock
arm 컨트롤러	JointGroupPositionController	JointTrajectoryController
servo 출력	/panda_arm_controller/commands (Float64MultiArray)	/panda_arm_controller/joint_trajectory (JointTrajectory)
move_group plan & execute X (arm)	X	✓
본 절차 적용	✓	✓

§ 4 의 확인 명령에서 마지막 토픽 이름만 바꿔 쓰면 된다.

관련 문서

- [external_input_adapters.md](#) — 어댑터 계약 (토픽·stamp·주기)
- [leader_follower_mirroring_design.md](#) — retarget 설계와 안전장치
- [../moveit/servo_client_programmers_guide.md](#) — servo 서비스·상태 코드
- [../replay/replay_mock_stack_guide.md](#) — 같은 제약이 재생 경로에도 적용된다
- [../src/rdfp/launch/README.md](#) — 런치 인벤토리